

miniFlink

Декларативная потоковая обработка данных на OCaml — учебная реализация ключевых концепций Apache Flink.

miniFlink выражает семантику потоковой обработки максимально прозрачно: весь конвейер мониторинга читается как спецификация, без явного `key_by`, без аннотаций типов, без ручных проверок на `null`.

```
let pipeline source =
  source
  |> Mf_event.with_watermarks ~latency:(seconds 3)
  |> Pipe.enrich (module Telemetry)
    ~from:devices
    ~merge:(fun t dev -> { t with device = dev })
  |> Pipe.window (module Telemetry) (Pipe.tumbling (seconds 30))
  |> Pipe.aggregate Rules.compute
  |> Pipe.flat_map (Rules.check Rules.fleet)
  |> Pipe.dedup (module Alert)
    ~rule:(fun a -> a.rule)
    ~cooldown:(minutes 5)
```

Обзор архитектуры

Проект разбит на ортогональные слои. Каждый слой зависит только от нижележащих; ядро никогда не импортирует слои надёжности или параллелизма.

Ядро

Никогда не меняется, без внешних зависимостей.

- `Stream` — pull-based поток `unit -> 'a option`. Стек вызовов = граф операторов.
- `Mf_event` — событие `Data | Watermark | Retract`. Watermark-стратегия.
- `Time` — единицы времени: `seconds`, `minutes`, `hours`.
- `Keyed` — type class `KEYED`: ключ группировки описывается один раз в типе.
- `Pipe` — операторы: `enrich`, `window`, `aggregate`, `dedup`, `flat_map`.
- `Table` — таблицы `of_list`, `of_stream`, `of_stream_ttl` для join.
- `Codec` — абстракция сериализации JSON / Protobuf.

Домен

- `Domain` — доменные типы с `@@deriving yojson, show`.
- `Rules` — бизнес-логика как чистые функции.

Параллелизм

Шардирование по ключу: `hash(key) mod N` направляет событие воркеру.

- `Channel` — канал между операторами с backpressure (bounded).
- `Parallel` — data-parallel выполнение. `Thread` (OCaml 4) / `Domain` (OCaml 5).
- `Checkpoint_parallel` — exactly-once: barrier-инжекция + per-worker snapshot.

Production-слои

Каждый: контракт (`.mli`) + `_noop` (нулевой overhead) + `_default` (production).

- `Dlq` — dead letter queue: ошибки decode не роняют конвейер.
- `Shutdown` — graceful shutdown: `SIGTERM` → `drain` → `exit`.
- `Metrics` — Prometheus-совместимые метрики + HTTP endpoint.
- `Barrier` — координатор Chandy-Lamport для распределённого снапшота.
- `State_backend` — хранилище стейта: `memory` / `RocksDB` (через C FFI).
- `Schema` — эволюция схемы.
- `Harness` — детерминированный тестовый фреймворк.
- `Runtime` — сборка всех слоёв по режиму (`Noop | Log | Parallel | Prod`).

С чего начать

Тutorial с пошаговым построением конвейера: [tutorial](#).

Запуск

```
dune build          # сборка (channel/parallel выбираются по версии OCaml)
dune exec bin/main.exe  # демо
dune test           # 13 тест-скит
dune exec bench/soak.exe -- 2000000 # soak: проверка отсутствия утечек
```

Гарантии и границы

Реализовано: event time, watermarks (с периодической эмиссией и финальным flush), tumbling/sliding окна, stateful-операторы, exactly-once стейта в параллельном режиме (barrier + recovery), table/join с TTL-eviction, retractions для поздних данных, персистентный RocksDB-стейт.

Сознательно вне области (*non-goals*): распределённое выполнение, remote shuffle, end-to-end exactly-once через внешние source/sink, incremental checkpointing. «mini» в названии честное — проект про *семантику*, не про распределённую инфраструктуру.

Module Stream

Pull-based ленивый поток.

Поток — это функция `unit -> 'a option`: каждый вызов возвращает следующий элемент, `None` означает конец. Стек вызовов операторов *и есть* граф обработки — никаких колбэков и промежуточных буферов (кроме `flat_map`, где буфер необходим).

```
type 'a t = unit -> 'a option
```

Тип потока элементов `'a`.

```
val of_list : 'a list -> unit -> 'b option
```

```
val empty : 'a t
```

```
val map : ('a -> 'b) -> (unit -> 'a option) -> unit -> 'b option
```

```
val filter : ('a -> bool) -> (unit -> 'b option) -> unit -> 'c option
```

```
val flat_map : ('a -> 'b list) -> (unit -> 'c option) -> unit -> 'd option
```

```
val iter : ('a -> unit) -> (unit -> 'b option) -> unit
```

```
val fold : ('a -> 'b -> 'c) -> 'd -> (unit -> 'e option) -> 'f
```

```
val to_list : (unit -> 'a option) -> 'b list
```

Module Time

```
type t = int
```

```
val ms : 'a -> 'b
```

```
val seconds : int -> int
```

```
val minutes : int -> int
```

```
val hours : int -> int
```

Module Mf_event

Событие потока: данные, watermark или отмена.

Все три конструктора текут по одному потоку и обрабатываются операторами через pattern matching — единообразно и с проверкой полноты компилятором.

```
type 'a t =  
  | Data of 'a * Time.t  
  | Watermark of Time.t  
  | Retract of 'a * Time.t
```

значение + его event-time

граница: события раньше уже пришли

отмена ранее выданного результата

Событие со значением 'a.

```
val data : 'a -> Time.t -> 'b t
```

```
val retract : 'a -> Time.t -> 'b t
```

```
val wm : Time.t -> 'a t
```

```
val value : 'a t -> 'b option
```

```
val ts : 'a t -> Time.t
```

```
val is_data : 'a t -> bool
```

```
val map_value : ('a -> 'b) -> 'c t -> 'd t
```

```
val with_watermarks_ext :  
  latency:int ->  
  ?interval:int ->  
  'a t Stream.t ->  
  'a0 t Stream.t
```

`with_watermarks_ext ~latency ?interval src` вставляет watermark-маркеры в поток. `Watermark = max_seen - latency`: все события с меньшим timestamp считаются прибывшими.

- `~latency` — допуск на опоздание (out-of-orderness).
- `~interval` — минимальный сдвиг watermark перед новой эмиссией (по event-time). При `interval=0` watermark эмитится на каждый новый максимум; больший `interval` уменьшает их число.

На конце потока эмитится финальный `watermark = max_seen`, чтобы закрыть все хвостовые окна.

```
val with_watermarks : latency:int -> 'a t Stream.t -> 'a t Stream.t
```

`with_watermarks ~latency src` — то же что `with_watermarks_ext` с `~interval:0` (watermark на каждый новый максимум). Обратно совместимая форма.

```
val pp_ts : int -> string
```

Module Keyed

```
module type S = sig ... end
```

```
module Make (T : sig ... end) : sig ... end
```

Module Table

```
type ('k, 'v) t = 'k -> 'v option
```

```
val of_list : ('a * 'b) list -> 'c -> 'd option
```

```
val of_hashtbl : ('a, 'b) Stdlib.Hashtbl.t -> 'a -> 'b option
```

```
val of_stream : key:(('a -> 'b) -> 'a0 Mf_event.t Stream.t -> 'c -> 'a1 option
```

```
val of_stream_ttl :  
  key:(('a -> 'b) ->  
  ttl:int ->  
  'a0 Mf_event.t Stream.t ->  
  'c ->  
  'a1 option
```

Module Pipe

Операторы конвейера: `enrich`, `window`, `aggregate`, `dedup`, `flat_map`.

Ключевая особенность: `window`, `enrich`, `dedup` получают ключ группировки из модуля `Keyed.S`, переданного первым аргументом — в пользовательском коде нет повторяющихся `~key:` параметров.

`window` использует locally abstract type (`type a`), что устраняет рекурсивный тип `α = string × α list` без `Obj.magic`.

```
val emap :
  ('a -> 'b) ->
  (unit -> 'a Mf_event.t option) ->
  unit ->
  'b Mf_event.t option
```

```
val efilt :
  ('a -> bool) ->
  (unit -> 'b Mf_event.t option) ->
  unit ->
  'b Mf_event.t option
```

```
val eflatmap :
  ('a -> 'b list) ->
  (unit -> 'c Mf_event.t option) ->
  unit ->
  'd Mf_event.t option
```

```
val enrich :
  (module Keyed.S with type t = 'a) ->
  from:(string, 'b) Table.t ->
  merge:(('a0 -> 'b0 option -> 'a1) ->
  'a2 Mf_event.t Stream.t ->
  'a3 Mf_event.t Stream.t
```

`enrich (module K) ~from ~merge upstream` обогащает каждое событие данными из справочной таблицы `from` по ключу `K.key`. Left join: если ключа нет в таблице, `merge` получает `None` и событие проходит небогащённым — конвейер не падает.

```
module WMap : sig ... end
```

```
type win_spec =
  | Tumbling of Time.t
  | Sliding of Time.t * Time.t
```

```
val assign : win_spec -> int -> (int * int) list
```

```
type 'a win_state =
  | Open of 'a list
  | Fired of 'a list
```

```
val window :
  (module Keyed.S with type t = 'a) ->
  ?latency:int ->
  ?allowed_lateness:int ->
```

```
win_spec ->
'a0 Mf_event.t Stream.t ->
(string * 'a1 list) Mf_event.t Stream.t
```

`window (module K) ?latency ?allowed_lateness spec upstream` группирует события по ключу `K.key` и временным окнам `spec` (`tumbling` или `sliding`). Окно закрывается когда `watermark` проходит его правую границу (плюс `latency`), выдавая (ключ, событие `list`).

Поздние данные в пределах `~allowed_lateness` переоткрывают закрытое окно: эмитится `Retract` старого результата, затем `Data` нового. За пределами `allowed_lateness` окно окончательно удаляется (ограничивает рост состояния).

```
val aggregate :
('a -> 'b -> 'c) ->
(unit -> ('d * 'e) Mf_event.t option) ->
unit ->
'f Mf_event.t option
```

`aggregate f` сворачивает события каждого окна (`key`, `vs`) в один результат `f key vs` (например `max`-скорость, `min`-топливо).

```
val stateful :
init:'a ->
f:(('b -> 'c Mf_event.t -> 'd * 'e Mf_event.t list) ->
(unit -> 'c Mf_event.t option) ->
unit ->
'e Mf_event.t option
```

```
val dedup :
(module Keyed.S with type t = 'a) ->
rule:(('a0 -> string) ->
cooldown:Time.t ->
'a1 Mf_event.t Stream.t ->
'a2 Mf_event.t Stream.t
```

`dedup (module K) ~rule ~cooldown upstream` подавляет повторные события с одинаковым `(K.key, rule)` в пределах окна `cooldown`.

Состояние ограничено: при каждом `watermark` записи старше `wm - cooldown` удаляются — они уже не могут подавить будущие события, поэтому удаление безопасно.

```
val map :
('a -> 'b) ->
(unit -> 'a Mf_event.t option) ->
unit ->
'b Mf_event.t option
```

```
val filter :
('a -> bool) ->
(unit -> 'a Mf_event.t option) ->
unit ->
'a Mf_event.t option
```

```
val flat_map :
('a -> 'b list) ->
(unit -> 'a Mf_event.t option) ->
unit ->
'b Mf_event.t option
```

```
val sink : ('a -> unit) -> (unit -> 'b Mf_event.t option) -> unit
```

```
val collect : (unit -> 'a Mf_event.t option) -> 'b list
```

```
val tumbling : Time.t -> win_spec
```

Неперекрывающиеся окна фиксированного размера.

```
val sliding : Time.t -> Time.t -> win_spec
```

Перекрывающиеся окна: `sliding size step` — окна размера `size` с шагом `step` (при `step < size` окна перекрываются).

```
val with_counter :  
  (unit -> unit) ->  
  (unit -> 'a Mf_event.t option) ->  
  unit ->  
  'a Mf_event.t option
```

Обернуть stream: вызывать `f()` на каждом Data событии

```
val window_instrumented :  
  (module Keyed.S with type t = 'a) ->  
  ?latency:int ->  
  observe_window_ms:(float -> unit) ->  
  win_spec ->  
  'a0 Mf_event.t Stream.t ->  
  (string * 'a1 list) Mf_event.t Stream.t
```

window с histogram для latency закрытия окна

Module Codec

```
type 'a t = {  
  encode : 'a -> bytes;  
  decode : bytes -> ('a, string) Stdlib.result;  
  name : string;  
}
```

```
val json :  
  encode:('a -> Yojson.Safe.t) ->  
  decode:(Yojson.Safe.t -> ('b, string) Stdlib.result) ->  
  'c t
```

```
val protobuf : encode:('a -> bytes) -> decode:(bytes -> 'b) -> 'a t
```

Module Domain

```
type position = {  
  lat : float;  
  lon : float;  
}
```

```
val position_to_yojson : position -> Yojson.Safe.t
```

```
val position_of_yojson :  
  Yojson.Safe.t ->  
  position Ppx_deriving_yojson_runtime.error_or
```

```
val pp_position :  
  Ppx_deriving_runtime.Format.formatter ->  
  position ->  
  Ppx_deriving_runtime.unit
```

```
val show_position : position -> Ppx_deriving_runtime.string
```

```
type device_info = {  
  owner : string;  
  max_speed : float;  
  zone : string;  
}
```

```
val device_info_to_yojson : device_info -> Yojson.Safe.t
```

```
val device_info_of_yojson :  
  Yojson.Safe.t ->  
  device_info Ppx_deriving_yojson_runtime.error_or
```

```
val pp_device_info :  
  Ppx_deriving_runtime.Format.formatter ->  
  device_info ->  
  Ppx_deriving_runtime.unit
```

```
val show_device_info : device_info -> Ppx_deriving_runtime.string
```

```
type telemetry = {  
  device_id : string;  
  speed_kmh : float;  
  fuel_pct : float;  
  position : position;  
  ts : int;  
  device : device_info option;  
}
```

```
val telemetry_to_yojson : telemetry -> Yojson.Safe.t
```

```
val telemetry_of_yojson :
```

```
Yojson.Safe.t ->  
telemetry Ppx_deriving_yojson_runtime.error_or
```

```
val pp_telemetry :  
  Ppx_deriving_runtime.Format.formatter ->  
  telemetry ->  
  Ppx_deriving_runtime.unit
```

```
val show_telemetry : telemetry -> Ppx_deriving_runtime.string
```

```
module Telemetry : sig ... end
```

```
type stats = {  
  device_id : string;  
  max_speed : float;  
  avg_speed : float;  
  min_fuel : float;  
  count : int;  
  device : device_info option;  
}
```

```
val stats_to_yojson : stats -> Yojson.Safe.t
```

```
val stats_of_yojson :  
  Yojson.Safe.t ->  
  stats Ppx_deriving_yojson_runtime.error_or
```

```
val pp_stats :  
  Ppx_deriving_runtime.Format.formatter ->  
  stats ->  
  Ppx_deriving_runtime.unit
```

```
val show_stats : stats -> Ppx_deriving_runtime.string
```

```
module Stats : sig ... end
```

```
type severity =  
  | Critical  
  | Warning  
  | Info
```

```
val severity_to_yojson : severity -> Yojson.Safe.t
```

```
val severity_of_yojson :  
  Yojson.Safe.t ->  
  severity Ppx_deriving_yojson_runtime.error_or
```

```
val pp_severity :  
  Ppx_deriving_runtime.Format.formatter ->  
  severity ->  
  Ppx_deriving_runtime.unit
```

```
val show_severity : severity -> Ppx_deriving_runtime.string
```

```
type alert = {  
  id : string;  
  device_id : string;  
  rule : string;  
  severity : severity;  
  message : string;  
  ts : int;  
}
```

```
val alert_to_yojson : alert -> Yojson.Safe.t
```

```
val alert_of_yojson :  
  Yojson.Safe.t ->  
  alert Ppx_deriving_yojson_runtime.error_or
```

```
val _ : Yojson.Safe.t -> alert Ppx_deriving_yojson_runtime.error_or
```

```
val pp_alert :  
  Ppx_deriving_runtime.Format.formatter ->  
  alert ->  
  Ppx_deriving_runtime.unit
```

```
val show_alert : alert -> Ppx_deriving_runtime.string
```

```
module Alert : sig ... end
```

```
val telemetry_json : telemetry Codec.t
```

```
val alert_json : alert Codec.t
```

Module Rules

```
val compute : string -> Domain.telemetry list -> Domain.stats
```

```
val alert :  
  rule:string ->  
  severity:Domain.severity ->  
  msg:(Domain.stats -> string) ->  
  Domain.stats ->  
  Domain.alert
```

```
val speed_limit : Domain.stats -> Domain.alert list
```

```
val fuel_level : Domain.stats -> Domain.alert list
```

```
val fleet : (Domain.stats -> Domain.alert list) list
```

```
val check :  
  (Domain.stats -> Domain.alert list) list ->  
  Domain.stats ->  
  Domain.alert list
```

Module Channel

Канал между операторами с backpressure.

Используется для передачи событий от dispatcher к воркерам в параллельном режиме. Bounded-канал даёт backpressure автоматически: producer блокируется когда канал полон, consumer — когда пуст.

Две реализации выбираются по версии OCaml (через dune-правило):

- **OCaml 4:** Mutex + Condition (Thread);
- **OCaml 5:** Atomic lock-free SPSC ring buffer (Domain).

Sentinel `None` из `pop` означает что канал закрыт и пуст.

```
type 'a t
```

Непрозрачный тип канала элементов `'a`.

```
val make_unbounded : unit -> 'a t
```

Создать неограниченный канал (без backpressure).

```
val make_bounded : int -> 'a t
```

Создать ограниченный канал заданной ёмкости (с backpressure).

```
val push : 'a t -> 'a -> unit
```

Записать значение. Блокирует если канал bounded и полон.

```
val try_push : 'a t -> 'a -> bool
```

Как `push`, но возвращает `false` если канал закрыт (например, consumer-воркер упал), вместо блокировки навсегда. Предотвращает deadlock при падении воркера.

```
val pop : 'a t -> 'a option
```

Прочитать значение. Блокирует если канал пуст и не закрыт. `None` — канал закрыт и пуст.

```
val try_pop : 'a t -> 'a option
```

Неблокирующее чтение. `None` — нет данных *сейчас*.

```
val close : 'a t -> unit
```

Закрыть канал. После этого `pop` вернёт оставшиеся элементы, затем `None`.

```
val length : 'a t -> int
```

Текущее число элементов в канале.

```
val to_stream : 'a t -> 'a Stream.t
```

Представить канал как pull-поток `Stream.t`.

Module Parallel

```
type worker_stats = {  
  mutable processed : int;  
  mutable emitted : int;  
  mutable wm_seen : int;  
}
```

```
val make_stats : unit -> worker_stats
```

```
val hash_key : string -> int -> int
```

```
val run_parallel_simple :  
  ?sink_factory:(int -> 'b -> unit) ->  
  workers:int ->  
  capacity:int ->  
  key_of:('a -> string) ->  
  pipeline:('a Mf_event.t Stream.t -> 'b Mf_event.t Stream.t) ->  
  source:'a Mf_event.t Stream.t ->  
  sink:('b -> unit) ->  
  unit ->  
  unit
```

Data-parallel pipeline. Шардирует source по hash(key_of) на N воркеров. Каждый воркер запускает свою копию pipeline. Sink защищён мьютексом (v4) или вызывается per-domain (v5).

Module Checkpoint_parallel

```
type epoch = int
```

```
type offset = int
```

Позиция чтения в источнике. Для Kafka — offset партии, для списка — индекс, для файла — байтовое смещение.

```
type 'a seekable_source = {  
  pull : unit -> 'a Mf_event.t option;  
  position : unit -> offset;  
  seek : offset -> unit;  
}
```

Адресуемый (seekable) источник: помимо чтения событий умеет сообщать текущую позицию и перематываться на сохранённую. Это то что делает возможным exactly-once на входной стороне — после сбоя мы перематываем источник на offset последнего закомиченного checkpoint и переобрабатываем без пропусков.

```
val seekable_of_list : 'a Mf_event.t list -> 'a0 seekable_source
```

Обернуть обычный список в seekable source (для тестов и in-memory).

```
type 'a msg =  
  | Event of 'a Mf_event.t  
  | Barrier of epoch
```

```
type worker_snapshot = {  
  worker : int;  
  epoch : epoch;  
  state : bytes;  
  processed : int;  
}
```

```
type checkpoint = {  
  cp_epoch : epoch;  
  cp_offset : offset;  
  cp_snapshots : worker_snapshot array;  
}
```

Полная запись checkpoint: epoch, позиция источника на момент barrier, и снапшоты всех воркеров. Source offset — ключевое добавление: без него восстановление не знает откуда читать.

```
type checkpoint_store = {  
  mutable committed : checkpoint list;  
  cs_mu : Mutex.t;  
  cs_persist : (checkpoint -> unit) option;  
}
```

```
val make_store : ?persist:(checkpoint -> unit) -> unit -> checkpoint_store
```

```
val commit : checkpoint_store -> checkpoint -> unit
```

```
val latest_checkpoint : checkpoint_store -> checkpoint option
```

```
val checkpoint_count : checkpoint_store -> int
```

```
type coordinator = {  
  workers : int;  
  store : checkpoint_store;  
  pending : worker_snapshot option array;  
  co_mu : Mutex.t;  
  co_done : Condition.t;  
}
```

```
val make_coordinator : workers:int -> store:checkpoint_store -> coordinator
```

```
val submit_snapshot : coordinator -> worker_snapshot -> unit
```

```
val wait_committed : coordinator -> epoch:epoch -> unit
```

```
val hash_key : string -> int -> int
```

```
type 'b transactional_sink = {  
  ts_write : epoch -> 'b -> unit;  
  ts_commit : epoch -> unit;  
  ts_abort : epoch -> unit;  
  ts_flush : unit -> unit;  
}
```

Sink с поддержкой two-phase commit, привязанного к checkpoint. Между barrier результаты пишутся в pre-commit (невидимы наружу); когда checkpoint зафиксирован — commit делает их видимыми; при сбое до commit — abort откатывает.

Это замыкает exactly-once на выходной стороне: переобработка после восстановления не создаёт дублей, потому что незакоммиченные результаты откатываются.

Для sink не умеющих транзакции используйте идемпотентный путь: `idempotent_sink` — детерминированный ключ + upsert.

```
val idempotent_sink : ('b -> unit) -> 'b0 transactional_sink
```

Идемпотентный sink: оборачивает обычную upsert-функцию. Commit/abort — no-op, потому что повторная запись по тому же детерминированному ключу безвредна (upsert перезапишет тем же). Самый дешёвый путь к корректному выходу, работает для любого sink с ключевой записью.

```
val buffered_sink : ('b list -> unit) -> 'b0 transactional_sink
```

Буферизующий транзакционный sink: накапливает результаты epoch, публикует пачкой на commit, отбрасывает на abort. Эмулирует 2PC для sink без нативных транзакций (in-memory pre-commit буфер).

```
val run_exactly_once :  
  workers:int ->  
  capacity:int ->  
  checkpoint_every:int ->  
  key_of:('a -> string) ->  
  make_state:(unit -> State_backend_memory.t) ->  
  process:(State_backend_memory.t -> 'a0 Mf_event.t -> 'b list) ->  
  source:'a1 seekable_source ->  
  sink:'b0 transactional_sink ->  
  store:checkpoint_store ->  
  unit ->
```

```
unit
```

`run_exactly_once` — data-parallel выполнение с полным end-to-end exactly-once:

- `source offset` фиксируется в каждом checkpoint (входная сторона);
- транзакционный `sink` коммитит результаты epoch только после того как checkpoint этого epoch зафиксирован (выходная сторона).

Параметры:

- `workers`, `capacity`, `checkpoint_every` — как раньше;
- `key_of` — ключ шардирования;
- `make_state` — создать backend воркера (вызывается N раз);
- `process backend epoch ev` — обработать событие, вернуть выходы;
- `source` — `seekable_source` (умеет position/seek);
- `sink` — `transactional_sink`;
- `store` — куда коммитить checkpoint-ы.

Воркер пишет выходы через `ts_write epoch`. Когда `barrier(epoch)` собран со всех воркеров и checkpoint зафиксирован, координатор вызывает `ts_commit epoch` — результаты становятся видимыми атомарно вместе с фиксацией стейта и offset.

```
val restore_latest :  
  checkpoint_store ->  
  State_backend_memory.t array ->  
  offset option
```

Раздать снимки последнего checkpoint воркерам. `backends.(i)` восстанавливается из snapshot воркера `i`. Возвращает offset с которого нужно перечитывать источник, либо `None` если checkpoint-ов нет.

```
val recover :  
  workers:int ->  
  make_state:(unit -> State_backend_memory.t) ->  
  source:'a seekable_source ->  
  checkpoint_store ->  
  State_backend_memory.t array
```

Протокол холодного старта после сбоя.

Единый путь восстановления: 1. прочитать последний валидный checkpoint из store; 2. восстановить стейт всех воркеров из снимков; 3. перемотать источник на сохранённый offset; 4. вернуть `backends` готовые к продолжению обработки.

Если checkpoint-ов нет (первый запуск) — `backends` пустые, источник остаётся в начале.

```
val serialize_checkpoint : checkpoint -> bytes
```

Сериализация checkpoint в bytes (Marshal). `worker_snapshot.state` уже bytes, поэтому Marshal записи безопасен.

```
val deserialize_checkpoint : bytes -> checkpoint
```

```
val durable_store : dir:string -> checkpoint_store
```

Создать store с durable-записью на диск. Каждый коммит пишет checkpoint в файл `dir/checkpoint_<epoch>.cp` плюс обновляет `dir/LATEST` с номером последнего epoch. Переживает рестарт процесса и (если `dir` на сетевом томе) смерть локального диска.

```
val load_durable : dir:string -> checkpoint_store
```

Загрузить последний durable checkpoint с диска в новый store (после рестарта процесса). Читает `dir/LATEST`, затем соответствующий `checkpoint_<epoch>.cp`.

Module Runtime

```
type mode =  
  | Noop  
  | Log  
  | Parallel  
  | Prod
```

```
type config = {  
  mode : mode;  
  parallelism : int;  
  capacity : int;  
  metrics_port : int;  
}
```

```
val noop : config
```

```
val log_cfg : config
```

```
val parallel : config
```

```
val prod : config
```

```
val make_dlq : mode -> (Dlq_log.entry -> unit) * (unit -> unit) * (unit -> int)
```

```
val safe_decode :  
  send_dlq:(Dlq_noop.entry -> unit) ->  
  topic:string ->  
  codec:(bytes -> ('a, string) Stdlib.result) ->  
  attempt:int ->  
  bytes ->  
  'b option
```

```
type metrics = {  
  events_in : Metrics_log.counter;  
  events_out : Metrics_log.counter;  
  events_dlq : Metrics_log.counter;  
  watermarks : Metrics_log.counter;  
  wm_lag_ms : Metrics_log.gauge;  
  last_wm : Metrics_log.gauge;  
  max_event_ts : Metrics_log.gauge;  
}
```

```
val make_metrics : 'a -> string -> metrics
```

```
type noop_metrics = unit
```

```
val noop_metrics : 'a -> unit
```

```
val make_shutdown : mode -> bool Stdlib.ref * (unit -> bool)
```

```
val guarded_source :  
  running:bool Stdlib.ref ->  
  is_requested:(unit -> bool) ->  
  on_data:(Time.t -> unit) ->  
  on_wm:(Time.t -> unit) ->  
  (unit -> 'a Mf_event.t option) ->  
  unit ->  
  'a Mf_event.t option
```

```
val run :  
  ?label:string ->  
  config ->  
  key_of:('a -> string) ->  
  source:'a0 Mf_event.t Stream.t ->  
  pipeline:('a1 Mf_event.t Stream.t -> 'b Mf_event.t Stream.t) ->  
  sink:('b0 -> unit) ->  
  unit ->  
  unit
```

```
val make_stream_with_dlq :  
  config ->  
  topic:string ->  
  codec:(bytes -> ('a, string) Stdlib.result) ->  
  ts_of:('b -> Time.t) ->  
  (unit -> (string * bytes) option) ->  
  unit ->  
  'c Mf_event.t option
```

Module Harness

```
type ('a, 'b) t
```

Контекст теста: входная очередь + pipeline + выходной буфер

```
val create : ('a Mf_event.t Stream.t -> 'b Mf_event.t Stream.t) -> ('a, 'b) t
```

Создать контекст теста

```
val push : ('a, 'b) t -> 'a -> ts:int -> unit
```

Подать одно событие

```
val push_wm : ('a, 'b) t -> ts:int -> unit
```

Подать watermark

```
val push_all : ('a, 'b) t -> ('a * int) list -> unit
```

Подать список событий

```
val run : ('a, 'b) t -> 'b list
```

Закрыть входной поток и вычитать все выходные события

```
val run_data : ('a, 'b) t -> 'b list
```

Запустить и вернуть только Data значения

```
val run_all : ('a, 'b) t -> 'b Mf_event.t list
```

Запустить и вернуть все события включая Watermark/Retract

```
val expect_count : ('a, 'b) t -> int -> unit
```

Assertion helpers — бросают Failure с сообщением

```
val expect : ('a, 'b) t -> ('b -> bool) -> string -> unit
```

```
val expect_none : ('a, 'b) t -> unit
```

```
val ev : string -> int -> float -> float -> Domain.telemetry
```

Утилиты для построения тестовых событий

```
val evs : (string * int * float * float) list -> Domain.telemetry list
```

Module Barrier_default

```
type epoch = int
```

```
type coordinator = {  
  workers : int;  
  checkpoint_every : int;  
  mutable epoch : int;  
  ready : bool array;  
  mu : Mutex.t;  
  all_ready : Condition.t;  
}
```

```
val create : workers:int -> checkpoint_every:int -> coordinator
```

```
val worker_ready : coordinator -> worker:int -> epoch:'a -> unit
```

```
val wait_all_ready : coordinator -> epoch:'a -> unit
```

```
val current_epoch : coordinator -> int
```

```
val next_epoch : coordinator -> int
```

Module Barrier_noop

```
type epoch = int
```

```
type coordinator = {  
  mutable epoch : int;  
}
```

```
val create : workers:'a -> checkpoint_every:'b -> coordinator
```

```
val worker_ready : 'a -> worker:'b -> epoch:'c -> unit
```

```
val wait_all_ready : 'a -> epoch:'b -> unit
```

```
val current_epoch : coordinator -> int
```

```
val next_epoch : coordinator -> int
```

Module D1q_log

```
type entry = D1q_noop.entry = {  
  topic : string;  
  payload : bytes;  
  error : string;  
  ts : int;  
  attempt : int;  
}
```

```
type t = {  
  mutable n : int;  
  mu : Mutex.t;  
}
```

```
val create : unit -> t
```

```
val send : t -> entry -> unit
```

```
val flush : 'a -> unit
```

```
val count : t -> int
```

Module D1q_noop

```
type t = {  
  mutable n : int;  
}
```

```
type entry = {  
  topic : string;  
  payload : bytes;  
  error : string;  
  ts : int;  
  attempt : int;  
}
```

```
val create : unit -> t
```

```
val send : t -> 'a -> unit
```

```
val flush : 'a -> unit
```

```
val count : t -> int
```

Module Fixtures

```
val ev : string -> int -> float -> float -> Domain.telemetry Mf_event.t
```

```
val scenario_alerts : Domain.telemetry Mf_event.t list
```

Module Metrics_log

```
type counter = {  
  c_name : string;  
  c_labels : (string * string) list;  
  mutable c_val : int;  
  c_mu : Mutex.t;  
}
```

```
type gauge = {  
  g_name : string;  
  g_labels : (string * string) list;  
  mutable g_val : float;  
  g_mu : Mutex.t;  
}
```

```
type histogram = {  
  h_name : string;  
  h_labels : (string * string) list;  
  mutable h_count : int;  
  mutable h_sum : float;  
  mutable h_buckets : int array;  
  h_mu : Mutex.t;  
}
```

```
val buckets_us : int array
```

```
val all_counters : counter list Stdlib.ref
```

```
val all_gauges : gauge list Stdlib.ref
```

```
val all_histograms : histogram list Stdlib.ref
```

```
val reg_mu : Mutex.t
```

```
val pp_labels : (string * string) list -> string
```

```
val counter : name:string -> labels:(string * string) list -> counter
```

```
val gauge : name:string -> labels:(string * string) list -> gauge
```

```
val histogram : name:string -> labels:(string * string) list -> histogram
```

```
val incr : counter -> unit
```

```
val add : counter -> int -> unit
```

```
val set_gauge : gauge -> float -> unit
```

```
val observe : histogram -> float -> unit
```

```
val dump : unit -> string
```

```
val start_reporter : interval_s:int -> unit
```

```
val start_server : ?port:int -> unit -> unit
```

Module Metrics_noop

```
type counter = unit
```

```
type gauge = unit
```

```
type histogram = unit
```

```
val counter : name:'a -> labels:'b -> unit
```

```
val gauge : name:'a -> labels:'b -> unit
```

```
val histogram : name:'a -> labels:'b -> unit
```

```
val incr : 'a -> unit
```

```
val add : 'a -> 'b -> unit
```

```
val set_gauge : 'a -> 'b -> unit
```

```
val observe : 'a -> 'b -> unit
```

```
val dump : unit -> string
```

```
val start_reporter : interval_s:'a -> unit
```

Module Rocksdb_ffi

```
type db
```

```
val open_db : string -> db
```

```
val close_db : db -> unit
```

```
val put : db -> string -> bytes -> unit
```

```
val get : db -> string -> bytes option
```

```
val delete : db -> string -> unit
```

Module Schema_default

```
type version = int
```

```
type 'a t = {  
  enc : 'a -> bytes;  
  dec : bytes -> ('a, string) Stdlib.result;  
  ver : version;  
  migrate : (version -> bytes -> bytes) option;  
}
```

```
val make :  
  version:version ->  
  encode:('a -> bytes) ->  
  decode:(bytes -> ('b, string) Stdlib.result) ->  
  ?migrate:(version -> bytes -> bytes) ->  
  unit ->  
  'a t
```

```
val encode : 'a t -> 'b -> bytes
```

```
val decode : 'a t -> bytes -> ('b, string) Stdlib.result
```

```
val current_version : 'a t -> version
```

Module Schema_noop

```
type version = int
```

```
type 'a t = {  
  enc : 'a -> bytes;  
  dec : bytes -> ('a, string) Stdlib.result;  
  ver : version;  
}
```

```
val make :  
  version:version ->  
  encode:('a -> bytes) ->  
  decode:(bytes -> ('b, string) Stdlib.result) ->  
  ?migrate:'c ->  
  unit ->  
  'a t
```

```
val encode : 'a t -> 'b -> bytes
```

```
val decode : 'a t -> bytes -> ('a, string) Stdlib.result
```

```
val current_version : 'a t -> version
```

Module Shutdown_default

```
val requested : bool Stdlib.ref
```

```
val mu : Mutex.t
```

```
val cond : Condition.t
```

```
val request : unit -> unit
```

```
val is_requested : unit -> bool
```

```
val register : on_shutdown:(unit -> unit) -> unit
```

```
val wait : unit -> unit
```

Module Shutdown_noop

```
val register : on_shutdown:'a -> unit
```

```
val is_requested : unit -> bool
```

```
val wait : unit -> unit
```

```
val request : unit -> unit
```

Module State_backend_memory

```
type t = (string, bytes) Stdlib.Hashtbl.t
```

```
val create : unit -> ('a, 'b) Stdlib.Hashtbl.t
```

```
val get : ('a, 'b) Stdlib.Hashtbl.t -> 'c -> 'b option
```

```
val set : ('a, 'b) Stdlib.Hashtbl.t -> 'c -> 'd -> unit
```

```
val delete : ('a, 'b) Stdlib.Hashtbl.t -> 'c -> unit
```

```
val keys : ('a, 'b) Stdlib.Hashtbl.t -> 'c list
```

```
val size : ('a, 'b) Stdlib.Hashtbl.t -> int
```

```
val snapshot : ('a, 'b) Stdlib.Hashtbl.t -> bytes
```

```
val restore : (string, bytes) Stdlib.Hashtbl.t -> bytes -> unit
```

Module State_backend_noop

```
type t = unit
```

```
val create : unit -> unit
```

```
val get : 'a -> 'b -> 'c option
```

```
val set : 'a -> 'b -> 'c -> unit
```

```
val delete : 'a -> 'b -> unit
```

```
val keys : 'a -> 'b list
```

```
val size : 'a -> int
```

```
val snapshot : 'a -> bytes
```

```
val restore : 'a -> 'b -> unit
```

Module State_backend_rocksdb

```
type t = {  
  db : Rocksdb_ffi.db;  
  path : string;  
  mutable known : (string, unit) Stdlib.Hashtbl.t;  
}
```

```
val default_dir : string
```

```
val create_at : string -> t
```

```
val create : unit -> t
```

```
val get : t -> string -> bytes option
```

```
val set : t -> string -> bytes -> unit
```

```
val delete : t -> string -> unit
```

```
val keys : t -> string list
```

```
val size : t -> int
```

```
val close : t -> unit
```

```
val snapshot : t -> bytes
```

```
val restore : t -> bytes -> unit
```